

Lezione 19 - Design Pattern

Introduzione: dal controllo della concorrenza alla qualità del design

Fino a questo momento abbiamo analizzato meccanismi **tecnici** del linguaggio: thread, scheduler, lock, `synchronized`, `wait()` e `notify()`.

Abbiamo visto come il problema non sia solo *far funzionare* il codice, ma **garantire coerenza, controllo e correttezza anche in presenza di più attori concorrenti**.

Questo ci porta a un livello superiore di ragionamento.

Se con la sincronizzazione abbiamo imparato a controllare l'**accesso alle risorse**, ora ci spostiamo su un piano più astratto: come progettare correttamente le **strutture del sistema** prima ancora di scrivere il codice?

Qui entrano in gioco i **Design Pattern**.

Non riguardano singole istruzioni o keyword, ma:

- **organizzazione delle classi**
- **distribuzione delle responsabilità**
- **riduzione dell'accoppiamento**
- **aumento di flessibilità e riuso**

Design Pattern

Introduzione ai Pattern GoF

Quando si parla di Design Pattern, si fa riferimento in particolare ai pattern descritti dal gruppo chiamato **Gang of Four (GoF)**, autori del libro:

“Design Patterns: Elements of Reusable Object-Oriented Software” (1994)

I Design Pattern non sono pezzi di codice pronti, **ma soluzioni ricorrenti a problemi di progettazione ricorrenti**.

Non definiscono **come scrivere una singola istruzione**:

- **ma come strutturare le classi e le loro relazioni in modo corretto**.

Un problema di design

La fase di design di un sistema è quella cruciale tra:

- **Analisi**
- **Implementazione**

L'analisi segue metodologie abbastanza strutturate (diagrammi UML, requisiti funzionali, casi d'uso, ecc.).

Il design invece **non ha un inquadramento canonico rigido**.

Ed è proprio qui che nasce la difficoltà.

Durante il design:

- **si introducono elementi e oggetti concreti**
- **si definiscono ruoli e responsabilità**
- **si stabiliscono relazioni tra classi**
- si applicano principi come:
 - **indipendenza**
 - **flessibilità**
 - **riuso**
 - **estendibilità**
 - **basso accoppiamento**
 - **alta coesione**

Ma a differenza dell'analisi, **non esiste una procedura unica che garantisca automaticamente un buon risultato.**

Ed è qui che sorge la domanda centrale:

| Cosa si intende per buon design?

Cosa significa “buon design”?

Un sistema che funziona **non è necessariamente un sistema ben progettato.**

× **Un sistema può funzionare ed essere comunque:**

- **difficile da modificare**
- **fragile rispetto ai cambiamenti**
- **poco riusabile**

- **troppo accoppiato**
- **complesso da mantenere**

Il problema del cattivo design emerge quasi sempre nel tempo, non subito.
Quando arrivano nuove richieste o modifiche, un sistema mal progettato tende a “rompersi” o a diventare ingestibile.

Al contrario:

✓ **Un buon design invece deve:**

- **facilitare l'estensione senza modificare codice esistente (principio Open/Closed)**
- **ridurre le dipendenze tra classi**
- **separare responsabilità**
- **rendere il sistema comprensibile**
- **favorire il riuso di componenti**

Un buon design quindi **non riguarda solo il presente, ma soprattutto la capacità del sistema di evolvere nel tempo.**

Perché servono i Design Pattern?

Il design, a differenza dell'analisi, **non è guidato da una ricetta unica.**

Due sviluppatori possono progettare lo stesso sistema in modi completamente diversi:

- uno può creare classi fortemente accoppiate
- un altro può distribuire meglio le responsabilità
- uno può scrivere codice difficile da estendere
- un altro può progettare pensando già alla futura evoluzione

I Design Pattern nascono proprio per questo.

Servono a:

- **documentare soluzioni che si sono dimostrate efficaci nel tempo**
- **fornire un vocabolario comune tra sviluppatori**
- **evitare di reinventare ogni volta la soluzione a problemi già noti**

- **migliorare la qualità architeturale del software**

Non sono regole obbligatorie, né formule magiche.

Sono **strumenti concettuali**, frutto dell'esperienza accumulata nella progettazione di sistemi object-oriented complessi.

In sostanza:

I Design Pattern aiutano a trasformare il design da attività "intuitiva" a processo più consapevole e strutturato.

Le metriche

Affrontare un problema, in generale, significa risolverlo.

Ma **il modo in cui lo si risolve può fare la differenza.**

Due programmi possono produrre esattamente lo stesso risultato funzionale, ma non sono necessariamente equivalenti dal punto di vista qualitativo.

Uno può essere:

- **più efficiente**
- **più leggibile**
- **più facilmente modificabile**
- **più elegante dal punto di vista strutturale**

Ed è proprio qui che entra in gioco il concetto di *qualità del software*.

Nel caso del codice, esistono **metriche abbastanza oggettive** per valutarne la bontà:

- **prestazioni (tempo di esecuzione, consumo di memoria)**
- **complessità algoritmica**
- **manutenibilità**
- **leggibilità**
- **testabilità**

Questi aspetti, pur non essendo sempre misurabili in modo perfetto, possono essere analizzati con strumenti concreti (analisi della complessità, profiling, coverage dei test, ecc.).

In altre parole, sul singolo modulo o metodo possiamo avere indicatori relativamente oggettivi.

Il problema dell'architettura

Quando però saliamo di livello, passando dal singolo metodo o classe alla **struttura complessiva del sistema**, la situazione cambia radicalmente.

Non esistono parametri rigidi e universali per misurare la “bontà” dell'architettura.

Non possiamo dire:

“Questa architettura vale 8/10” applicando una formula matematica.

Possiamo però valutarla indirettamente osservando aspetti come:

- quanto è facile aggiungere nuove funzionalità
- quanto è semplice modificare il comportamento esistente
- quanto sono accoppiati i moduli
- quanto le responsabilità sono distribuite in modo coerente
- quanto il sistema è estendibile senza modificare codice già stabile

Questi criteri però sono **qualitativi**, non puramente numerici.

E soprattutto emergono nel tempo:

- spesso ci si accorge della qualità di un'architettura solo quando si prova ad evolverla.

Un'architettura mediocre funziona oggi, ma si rompe domani.

Un buon design invece regge nel tempo.

Il design è una forma d'arte?

A questo punto la domanda diventa naturale:

Il design è dunque una forma d'arte?

In parte sì.

Il design non è una semplice applicazione meccanica di regole.

Richiede:

- **esperienza**

- **sensibilità architettuale**
- **capacità di prevedere evoluzioni future**
- **capacità di bilanciare compromessi**
- **capacità di scegliere tra alternative tutte tecnicamente valide**

Ogni scelta architettuale è un compromesso tra:

- **flessibilità**
- **semplicità**
- **performance**
- **riuso**
- **complessità strutturale**

Tuttavia, **non è un'arte arbitraria o puramente intuitiva.**

Non si progetta “a sentimento”.

Ed è proprio qui che entrano in gioco i Design Pattern.

I pattern forniscono:

- **linee guida consolidate**
- **modelli riutilizzabili**
- **soluzioni ricorrenti a problemi ricorrenti**
- **un vocabolario comune tra sviluppatori**
- **strutture già validate dall'esperienza**

In questo modo, il design smette di essere solo “creatività personale” e diventa un'attività più consapevole, più strutturata e più trasferibile.

I pattern non impongono una soluzione, ma orientano le scelte.

☰ In sintesi:

- **Scrivere codice corretto è necessario.**
- **Scrivere codice ben progettato è ciò che rende il sistema sostenibile nel tempo.**
- **Le metriche aiutano a valutare il codice.**
- **I Design Pattern aiutano a strutturare l'architettura.**

Un processo induttivo

Per risolvere i “problemi” della vita quotidiana, il nostro cervello utilizza da sempre **deduzione logica e modelli mentali**.

Ma ciò che fa realmente la differenza è l'esperienza.

È l'“allenamento” continuo che ci permette di riconoscere situazioni già vissute e di reagire in modo più rapido ed efficace.

Il cervello umano infatti tende, spesso in modo inconscio, a identificare:

- **strutture simili**
- **schemi ricorrenti**
- **configurazioni già incontrate**

In altre parole, riconosce dei **pattern**.

Il principio è semplice:

una situazione “già vissuta”

- **se viene riconosciuta**
- **può essere gestita grazie all'esperienza maturata**

Non si inventa ogni volta una soluzione da zero.

Si riusa ciò che si è già dimostrato efficace.

Collegamento con i Design Pattern

I Design Pattern funzionano esattamente allo stesso modo.

Non sono invenzioni rivoluzionarie.

Non introducono concetti completamente nuovi.

Sono piuttosto:

- **modelli di progettazione consolidati nel tempo**
- **schemi ricorrenti che si sono dimostrati efficaci**
- **pratiche di buon senso formalizzate**

Non hanno la pretesa di “inventare” qualcosa.

Raccolgono e organizzano soluzioni che sviluppatori esperti hanno già utilizzato con successo in contesti differenti.

In questo modo forniscono:

- **punti di riferimento**
- **un linguaggio comune**
- **una guida per affrontare problemi simili**

☰ Un esempio intuitivo

Pensiamo agli algoritmi di ordinamento.

Quando dobbiamo ordinare una lista, non reinventiamo ogni volta il meccanismo di confronto e scambio.

Facciamo riferimento a modelli noti: ad esempio **Bubble Sort**, Merge Sort, Quick Sort.

Anche se non li abbiamo inventati noi, li utilizziamo perché:

- appartengono a una famiglia di problemi già studiata
- hanno una struttura riconoscibile
- hanno proprietà note

Allo stesso modo, nel design orientato agli oggetti, quando riconosciamo una certa struttura problematica, possiamo dire:

| “Questa situazione assomiglia a qualcosa che ho già visto.”

Ed è lì che entra in gioco il pattern.

Il progettista deve quindi:

- riconoscere la situazione
- capire a quale famiglia di problemi appartiene
- scegliere il pattern più adatto

Quindi il **meccanismo che porta alla nascita di un pattern è di tipo induttivo.**

Non si parte dalla teoria per poi applicarla alla pratica.

Si parte dalla pratica ripetuta e si generalizza.

Il processo è il seguente:

1. **Identifico una famiglia di problemi simili**
2. **Osservo che una certa soluzione funziona in tutti questi casi**
3. **Analizzo la soluzione e la astraggo dal contesto specifico**
4. **Formalizzo la struttura in modo riusabile**

Questo significa che un pattern non è solo una soluzione tecnica, ma è il risultato di:

- **osservazione**
- **esperienza**
- **astrazione**

Cosa comprende realmente un pattern?

Con il termine *pattern* spesso si pensa solo alla soluzione.

In realtà un pattern completo comprende tre elementi fondamentali:

- **Problema**
- **Soluzione**
- **Conseguenze**

Le conseguenze sono fondamentali perché ogni scelta architetturale comporta:

- **vantaggi**
- **svantaggi**
- **compromessi**

Un pattern non dice solo “come fare”, ma anche:

- **quando usarlo**
- **quando NON usarlo**
- **quali effetti produce sulla struttura del sistema**

☰ In sintesi:

I Design Pattern non sono formule magiche.

Sono il risultato di un processo induttivo basato sull'esperienza collettiva.

Il loro valore sta nel fatto che ci permettono di:

- **riconoscere strutture ricorrenti**
- **evitare di reinventare soluzioni già validate**
- **progettare in modo più consapevole**

E questo segna il passaggio da “scrivere codice che funziona” a “progettare sistemi che evolvono nel tempo”.

Ruolo dei pattern

Dal punto di vista delle architetture software, i Design Pattern non sono semplicemente “schemi eleganti”, ma **strumenti concreti che incidono sulla qualità complessiva del sistema.**

Essi:

- **diminuiscono i rischi**
- **incrementano la produttività**
- **aumentano la standardizzazione**
- **favoriscono un più alto livello di qualità**

Vediamo perché.

Diminuiscono i rischi

Come abbiamo già detto in precedenza, **quando si adotta un pattern, non si sta sperimentando una soluzione improvvisata.**

Si sta utilizzando:

- **una struttura già validata**
- **una soluzione che ha funzionato in contesti simili**
- **un approccio già discusso e analizzato nella comunità**

✓ **Questo riduce il rischio di:**

- **introdurre errori strutturali**
- **creare dipendenze eccessive**
- **compromettere l'evoluzione futura del sistema**

In altre parole, si riduce il rischio architetturale.

Incrementano la produttività

Un pattern evita di dover progettare ogni volta “da zero”.

Quando riconosci un problema e sai che esiste un pattern adatto:

- **non devi inventare una struttura**
- **non devi testare soluzioni improvvisate**
- **puoi concentrarti sull’adattamento al contesto specifico**

Questo rende il processo di design più rapido e più mirato.

La produttività aumenta non perché si scrive più codice, ma perché si prende una decisione progettuale più velocemente e con maggiore sicurezza.

Aumentano la standardizzazione

I pattern forniscono un linguaggio comune.

Se in un team si dice:

“Qui potremmo usare un Singleton”

oppure

“Questa situazione richiama un Observer”

non si sta solo nominando una tecnica, ma una struttura ben precisa.

Questo permette:

- **comunicazione più chiara**
- **minore ambiguità**
- **maggiore coerenza tra sviluppatori**

La standardizzazione non significa rigidità, ma condivisione di modelli comuni.

Favoriscono un più alto livello di qualità

Un’architettura costruita seguendo pattern consolidati tende ad avere:

- **basso accoppiamento**
- **alta coesione**

- maggiore estendibilità
- maggiore manutenibilità

✓ **Questo si traduce in sistemi che:**

- evolvono più facilmente
- resistono meglio ai cambiamenti
- risultano più comprensibili nel tempo

☰ **In sintesi:**

I Design Pattern non servono solo a “scrivere meglio il codice”.

Servono a progettare sistemi più solidi, più comunicabili e più sostenibili.

E questo li rende strumenti fondamentali nella fase di design, dove le decisioni prese oggi influenzeranno tutto il ciclo di vita del software.

Alexander: un architetto

Prima che i Design Pattern entrassero nell'ingegneria del software, il concetto nasce in **architettura**.

Il primo a formalizzarlo è stato **Christopher Alexander**, architetto, matematico e professore presso la University of California.

È noto per:

- la progettazione di edifici complessi in California, Giappone e Messico
- i suoi contributi teorici nel campo dell'architettura

Il suo libro più famoso è:

“A Pattern Language” (1977)

In questo testo Alexander descrive **253 pattern architettonici** che risolvono problemi ricorrenti nella progettazione di città, edifici e spazi abitativi.

Tra il 1977 e il 1979 introduce formalmente il concetto di **design pattern**.

L'idea chiave di Alexander

Alexander osserva che, nella progettazione architettonica, alcuni problemi si ripresentano continuamente.

Invece di reinventare ogni volta una soluzione, si possono identificare:

- **situazioni ricorrenti**
- **forze in conflitto**
- **soluzioni strutturali già sperimentate**

Un pattern, in architettura classica, è composto da tre elementi fondamentali:

1 Contesto

È la situazione problematica di progetto.

2 Problema

È l'insieme delle "forze" in gioco in quel contesto.

Le forze sono vincoli, esigenze, condizioni ambientali o funzionali che devono essere bilanciate.

3 Soluzione

È un insieme di forme, regole o configurazioni che permettono di risolvere il problema, bilanciando le forze.

Questa struttura:

Contesto → Problema → Soluzione

è esattamente la struttura che verrà poi adottata anche nei Design Pattern software.

☰ Esempio architettonico: il Patio

Vediamo un esempio concreto per capire meglio il concetto.

1. Contesto

Costruzione di una villetta.

2. Forze (problema)

- Si desidera disporre di uno spazio esterno.
- Tuttavia, piove spesso.

Qui le forze sono in conflitto:

- voglio uno spazio aperto
- ma devo proteggermi dalle intemperie

3. Soluzione

Realizzare un **patio**.

Il patio rappresenta una configurazione architettonica che:

- mantiene la relazione con l'esterno
- offre protezione parziale dagli agenti atmosferici

Non è un'invenzione casuale, ma una soluzione consolidata nel tempo. **Title**

Collegamento con i Design Pattern software

Il punto fondamentale è questo:

- **Alexander non inventa soluzioni nuove.**

Egli:

1. **osserva problemi ricorrenti**
2. **identifica soluzioni efficaci già applicate**
3. **le astrae dal contesto specifico**
4. **le formalizza come pattern**

Questo stesso approccio verrà adottato nella progettazione software.

Infatti, anche nei Design Pattern software troviamo sempre:

- **Contesto**
- **Problema**
- **Soluzione**
- **Conseguenze**

Il pattern non è codice pronto, ma una struttura concettuale riusabile.

Design pattern (in architettura)

Quindi, secondo **Christopher Alexander**, la definizione di pattern è:

descrive il nucleo di una soluzione relativa a un problema che compare frequentemente in un determinato contesto.

Ci sono tre elementi fondamentali dentro questa definizione:

- **esiste un contesto**
- **esiste un problema ricorrente**
- **esiste una struttura di soluzione**

Il pattern non è la soluzione completa e dettagliata.

È il modello della soluzione.

Il concetto chiave

Alexander afferma che un pattern deve essere strutturato in modo tale da poter:

“usare quella soluzione un milione di volte, senza mai farlo allo stesso modo”.

Questa frase è fondamentale.

Significa che:

- **il pattern non è uno schema rigido**
- **non è una copia identica da replicare**
- **non è un progetto prefabbricato**

È invece una **struttura adattabile**.

Ogni volta che lo si applica:

- **il contesto cambia leggermente**
- **le forze in gioco cambiano**
- **i vincoli cambiano**

ma il **principio strutturale** rimane valido.

Cosa implica questo concetto?

Un pattern:

- non fornisce dettagli concreti
- non impone un'unica implementazione
- non vincola la creatività

Fornisce invece:

- una linea guida strutturale
- una soluzione astratta
- un modello riusabile

È una forma di conoscenza condensata.

Collegamento con il software

Questo concetto sarà identico nei Design Pattern software.

Anche nel software:

- non si copia codice
- non si applica uno schema meccanico
- non si ottiene sempre lo stesso risultato

Si applica una **struttura concettuale**, che poi viene adattata al caso specifico.

Gamma: un progettista

Erich Gamma è un informatico svizzero, noto soprattutto come co-autore del libro **Design Patterns: Elements of Reusable Object-Oriented Software (in italiano "_Design Patterns - Elementi per il riuso di software ad oggetti")**.

Nel 1981 consegue un dottorato a Zurigo lavorando su tematiche legate ai pattern.

Nel 1995 ottiene grande notorietà nella comunità della programmazione orientata agli oggetti grazie alla pubblicazione del libro insieme al gruppo noto come **Gang of Four (GoF)**.

Ha contribuito in modo decisivo alla diffusione e alla formalizzazione del concetto di design pattern nel software.

È inoltre:

- principale progettista di **JUnit** (1998, insieme a Kent Beck)

- tra i principali responsabili dello sviluppo di Eclipse, piattaforma di sviluppo orientata agli oggetti

Un design Pattern(Gamma)

Secondo Gamma (e la GoF), un pattern software è:

- una soluzione provata
- ampiamente applicabile
- relativa a un particolare problema di progettazione
- descritta in forma standard
- riusabile facilmente

Definizione più formale:

“Descrizione di classi e oggetti comunicanti adatti a risolvere un problema progettuale generale in un contesto particolare”.

Quindi rispetto alla definizione di Alexander, qui attenzione si sposta su:

- classi
- oggetti
- interazioni tra oggetti

Il focus non è più solo la struttura astratta della soluzione, ma:

- il modo in cui le classi sono organizzate
- il modo in cui gli oggetti collaborano
- le responsabilità distribuite nel sistema

Un pattern quindi describe:

- ruoli
- responsabilità
- collaborazioni
- struttura delle relazioni

Non describe codice concreto.

Ma piuttosto, describe una configurazione di oggetti comunicanti.

Alexander parlava di:

- **nucleo di una soluzione riusabile infinite volte**

Gamma traduce questo concetto nel dominio software:

- **una soluzione tipica**
- **espressa in termini di classi e oggetti**
- **formalizzata in modo standard**
- **riapplicabile in contesti diversi**

Il Libro del GoF

Il libro **Design Patterns: Elements of Reusable Object-Oriented Software**

(1994), scritto da **Erich Gamma**, Richard Helm, Ralph Johnson e John Vlissides — noti come *[Gang of Four (GoF)]*

(https://it.wikipedia.org/wiki/Gang_of_Four(scrittori))_ — è considerato la “bibbia” dei design pattern.

È il primo testo che:

- **applica in modo sistematico il concetto di *design pattern* alla programmazione orientata agli oggetti**
- **individua e formalizza i principali pattern ricorrenti nell'OOP**
- **fornisce un linguaggio comune per descrivere soluzioni progettuali**

Il libro si presenta come un **catalogo di 23 pattern**, organizzati in tre categorie:

1. **Creazionali**
2. **Strutturali**
3. **Comportamentali**

Ogni pattern è descritto secondo una struttura standard che include:

- Nome
- Problema
- Soluzione
- Conseguenze

Questa formalizzazione è uno degli elementi più innovativi dell'opera:

- non viene descritto codice specifico, ma un modello riusabile di collaborazione tra classi e oggetti.

Origini e sviluppo storico

L'idea dei pattern non nasce nel software.

- **1977 – Origini nell'architettura**

L'architetto **Christopher Alexander** introduce il concetto di *pattern* nel libro *Pattern Language*, definendo i pattern come soluzioni ricorrenti a problemi progettuali in un determinato contesto.

- **Anni '80 – I pattern arrivano nel software**

Kent Beck e **Ward Cunningham** sperimentano l'uso dei pattern nella progettazione di interfacce grafiche in **Smalltalk**, trasferendo le idee di Alexander nell'ambito della programmazione.

- **1995 – Pubblicazione del libro GoF**

Con la pubblicazione del libro, i pattern vengono formalizzati in modo rigoroso e diventano un riferimento per la progettazione OO.

- **Anni '90 – Diffusione globale**

Nasce la Hillside Group, che organizza la conferenza PLoP (*Pattern Languages of Programs*). I design pattern entrano nei corsi universitari e nella formazione professionale.

- **Anni 2000–2010 – Oltre l'OOP**

Il concetto di pattern si estende dalle singole classi alle architetture software: MVC, SOA, pattern concorrenti, enterprise pattern (EJB, POJO).

- **2010–oggi – Pattern moderni**

Con cloud, container e microservizi emergono nuovi pattern architetturali come Circuit Breaker, Saga e CQRS.

Tipi di pattern

Le principali categorie di pattern software si distinguono in base al **livello di astrazione** a cui operano:

- **Pattern architetturali** (*stili architetturali*)
- **Pattern progettuali** (*micro-architetture*) → quelli catalogati dalla GoF
- **Idiomi** → soluzioni di basso livello, spesso legate a uno specifico linguaggio

La differenza fondamentale sta nella scala del problema che affrontano:

- più si sale di livello, più il pattern influenza la struttura complessiva del sistema.

Pattern architetturali

I pattern architetturali sono pattern di **alto livello**.

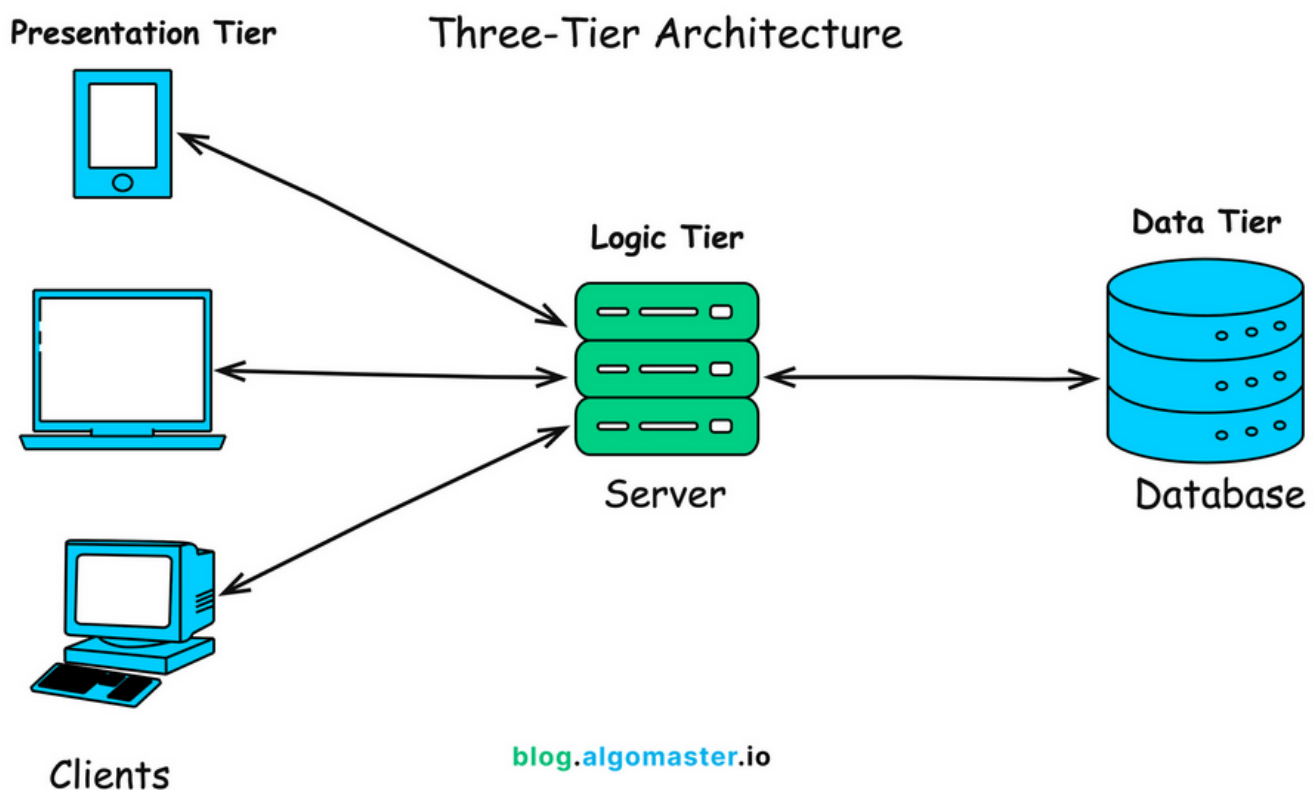
Non riguardano singole classi o oggetti, **ma guidano la decomposizione dell'intero sistema in sottosistemi, definendo:**

- ruoli e responsabilità delle parti
- regole e criteri di comunicazione reciproca
- modalità di distribuzione delle funzionalità

In altre parole, stabiliscono **come è organizzata l'architettura generale del software**.

Esempi di pattern architetturali

1. Client-Server

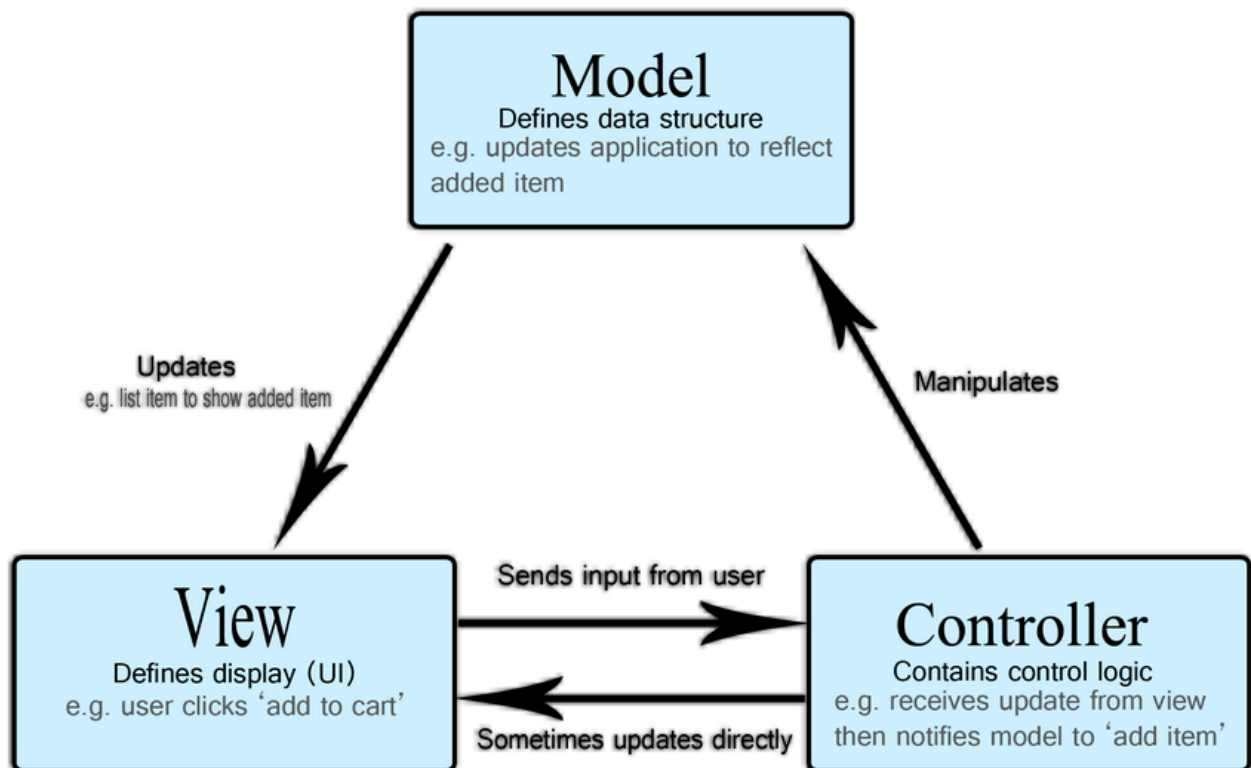


Il sistema è diviso in:

- **client** → richiede servizi
- **server** → fornisce servizi

È alla base del web e delle applicazioni distribuite.

2. Model-View-Controller (MVC)

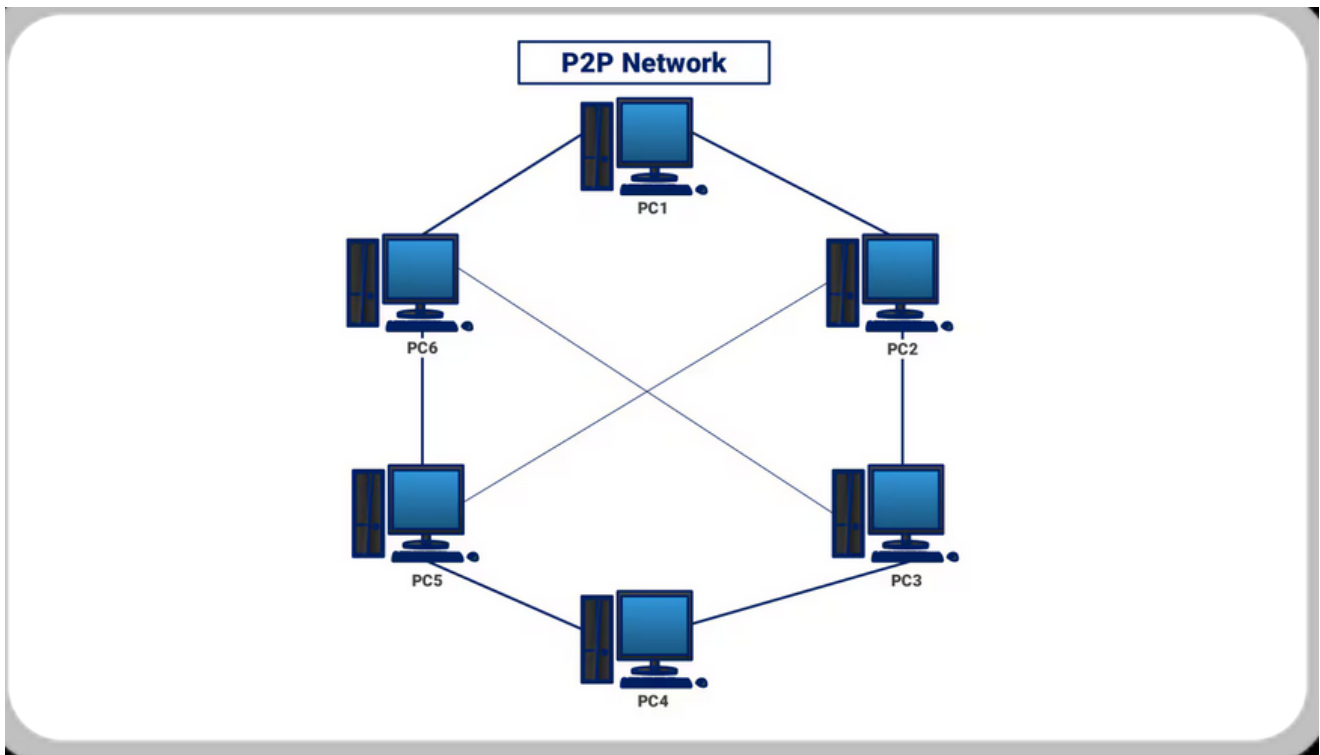


Separa il sistema in:

- **Model** → logica e dati
- **View** → interfaccia utente
- **Controller** → gestione input e coordinamento

Favorisce separazione delle responsabilità e manutenibilità.

3. Peer-to-Peer



In questa architettura:

- **Non esiste un server centrale.**
- **Ogni nodo può essere contemporaneamente client e server.**

È tipico dei sistemi distribuiti decentralizzati.

Quindi, in sintesi, i pattern architetturali:

- **operano a livello macro**
- **influenzano l'intero sistema**
- **definiscono la struttura portante dell'applicazione**

Successivamente, all'interno di un'architettura scelta, si applicheranno i **pattern progettuali (GoF)** per risolvere problemi più specifici tra classi e oggetti.

Pattern progettuali

I pattern progettuali sono quelli che comunemente identifichiamo come *design pattern*.

Operano a un livello più basso rispetto ai pattern architetturali:

- **non definiscono la struttura complessiva del sistema, ma intervengono nella progettazione di dettaglio dei singoli componenti.**

Per questo motivo vengono spesso definiti **micro-architetture**:

- **forniscono schemi ricorrenti per organizzare classi e oggetti e per raffinare le comunicazioni tra gli elementi di un sistema software.**

Il loro obiettivo principale è:

- **migliorare la distribuzione delle responsabilità**
- **ridurre l'accoppiamento**
- **rendere il codice più estensibile**
- **rendere esplicite le collaborazioni tra oggetti**

Esempi tipici sono i pattern catalogati nel libro della GoF:

- Factory Method
- Command
- Proxy
- Observer

Questi pattern non impongono una tecnologia specifica, **ma forniscono una struttura concettuale riutilizzabile che può essere adattata ai diversi contesti.**

Idiomi

Gli idiomi **rappresentano il livello più basso tra le categorie di pattern.**

Sono:

- **pattern di basso livello**
- **fortemente legati a uno specifico linguaggio di programmazione**
- **orientati all'implementazione concreta**

Mentre un design pattern descrive una soluzione concettuale indipendente dal linguaggio, un idioma spiega:

- **come realizzare quella soluzione sfruttando le caratteristiche di un determinato linguaggio.**

Gli idiomi descrivono quindi:

- **come implementare particolari elementi**
- **come modellare relazioni tra elementi**

- come sfruttare costrutti specifici del linguaggio (modificatori di accesso, keyword, gestione della memoria, ecc.)

Esempio: implementazione del Singleton in Java

Problema:

come garantire che una classe abbia una sola istanza?

In Java, un possibile **idioma** consiste nel:

1. **Definire un attributo** `private static` **chiamato** `instance` **del tipo della classe.**
2. **Implementare un metodo** `public static getInstance()` **che restituisce tale istanza.**
3. **Impostare il costruttore come** `private` **per impedire l'istanziamento dall'esterno.**

In questo caso:

- la struttura concettuale è il pattern Singleton,
- mentre il modo specifico in cui lo si implementa in Java rappresenta l'idioma.

☰ **Visione d'insieme**

Possiamo quindi distinguere chiaramente tre livelli:

- **Architetturali** → **organizzano l'intero sistema**
- **Progettuali (GoF)** → **organizzano classi e oggetti**
- **Idiomi** → **organizzano l'implementazione concreta nel linguaggio**

Questa stratificazione aiuta a comprendere che i pattern non operano tutti allo stesso livello, ma intervengono in fasi diverse del processo di progettazione software.

Pattern vs Idioma

Quindi, **la distinzione tra pattern e idioma chiarisce ulteriormente i diversi livelli a cui può operare una soluzione progettuale.**

1. Un **pattern** è:

- **legato al paradigma (ad esempio l'orientamento agli oggetti).**
- Propone un **modello generale**:
 - **cioè una soluzione astratta e riutilizzabile che non dipende da un linguaggio specifico.**
- **È una struttura concettuale che descrive**:
 - **come organizzare classi e oggetti per risolvere un certo tipo di problema.**

2. Un **idioma**, invece, è legato alla **tecnologia**.

- **Propone una soluzione concreta, specifica di un determinato linguaggio di programmazione, e sfrutta direttamente le sue caratteristiche sintattiche e semantiche.**

Possiamo quindi dire che:

- **il pattern fornisce la struttura logica della soluzione**
- **l'idioma fornisce la forma tecnica dell'implementazione**

Un design idiomatico, infatti, può rinunciare alla genericità tipica di un pattern per adottare una soluzione che sfrutta al massimo le potenzialità offerte dal linguaggio ([modificatori di accesso](#), keyword specifiche, gestione della concorrenza (es: [Multi-threading di java](#), ecc.).

In questo senso, pattern e idiomi non sono in contrapposizione, **ma operano su livelli diversi e complementari.**

Elementi chiave di un Design Pattern

Un design pattern non è semplicemente un'idea informale, ma **viene descritto secondo una struttura standardizzata**.

Questo schema descrittivo è uno degli aspetti che hanno reso il lavoro della GoF così influente.

Gli elementi fondamentali sono:

1 Nome del pattern

Il nome è molto più di un'etichetta:

- **diventa parte del vocabolario progettuale condiviso tra sviluppatori.**

 **Dire “applichiamo un Observer” o “usiamo una Factory”** permette di **comunicare rapidamente una struttura complessa senza doverla**

spiegare ogni volta nei dettagli.

Il nome rappresenta quindi:

- **un concetto ad alto livello di astrazione**
- **uno strumento di comunicazione tecnica**
- **un riferimento a una soluzione consolidata**

2 Problema

Descrive:

- **in quale contesto il pattern è applicabile**
- **quali sono le condizioni che rendono opportuna la sua adozione**
- **quali forze o vincoli sono in gioco**

Un pattern non è universale:

- **si applica solo quando si verificano determinate condizioni progettuali.**

🔗 Capire il problema è fondamentale per evitare un uso improprio del pattern.

3 Soluzione

La soluzione non è codice pronto all'uso, ma uno schema strutturale.

Specifica:

- **quali sono gli elementi partecipanti (classi, oggetti, interfacce)**
- **quali relazioni li collegano**
- **quali responsabilità assumono**
- **come collaborano tra loro**

🔗 Il livello è strutturale è concettuale:

- **non entra nei dettagli implementativi, che dipendono dal linguaggio e dal contesto.**

4 Conseguenze

Ogni pattern comporta costi e benefici.

Le conseguenze analizzano:

- **impatto su riusabilità**
- **impatto su estensibilità**
- **impatto su portabilità**
- **eventuale aumento di complessità**

🔔 **Un buon progettista non applica un pattern “perché sì”, ma valuta sempre le implicazioni strutturali della scelta.**

☰ In sintesi:

Possiamo quindi vedere un design pattern come **una unità strutturata di conoscenza progettuale**, composta da:

Nome + Problema + Soluzione + Conseguenze

Questa formalizzazione rende i pattern strumenti didattici, comunicativi e progettuali, e contribuisce a trasformare il design software da attività puramente intuitiva a disciplina basata su modelli condivisi e consolidati

Classificazione dei Pattern del GoF

I pattern del GoF vengono classificati secondo **due criteri fondamentali**:

1. **Scopo (Purpose)** → **cosa fanno**
2. **Campo di applicazione (Scope)** → **se operano principalmente su classi o su oggetti**

In base allo **scopo**, i pattern si dividono in tre grandi categorie:

- **Creational (5)** → **riguardano la creazione degli oggetti**
- **Structural (7)** → **riguardano la composizione e la struttura**
- **Behavioral (11)** → **riguardano la comunicazione e i comportamenti**

Pattern Creational (5)

Hanno l'obiettivo di:

- **astrarre e controllare il processo di creazione degli oggetti.**
- Factory Method
- Abstract Factory
- Builder
- Prototype
- Singleton

Pattern Structural (7)

Si occupano di:

- **come classi e oggetti vengono composti per formare strutture più grandi e flessibili.**
- Adapter (Class)
- Adapter (Object)
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Pattern Behavioral (11)

Descrivono **come gli oggetti comunicano tra loro e come vengono distribuite le responsabilità.**

- Interpreter
- Template Method
- Chain of Responsibility
- Command
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Visitor

GoF: Pattern Creational

Quindi abbiamo detto che i pattern creazionali hanno uno **scopo generale molto preciso**:

Astrarre l'istanziamento degli oggetti.

Significa che il sistema diventa indipendente dal modo in cui gli oggetti vengono creati.

Quindi, chi usa l'oggetto (client) non deve necessariamente sapere **come e quale classe concreta viene istanziata.**

Quando si usano?

I pattern creazionali sono particolarmente utili quando:

- **la creazione degli oggetti è complessa o variabile**
- **il client non deve conoscere la classe concreta**
- **è necessario controllare il numero di istanze (es. Singleton)**
- **la creazione deve essere centralizzata o configurabile**
- **il sistema deve essere facilmente estendibile con nuove varianti**

In sostanza, quando l'operazione `new` inizia a creare dipendenze troppo forti, è il momento di riflettere su un pattern creazionale.

✓ Vantaggi

1. Disaccoppiano creazione e utilizzo

- **Il client usa un oggetto senza sapere come è stato creato.**

2. Favoriscono l'estensibilità

- **È possibile introdurre nuove classi concrete senza modificare il codice esistente (principio Open/Closed).**

3. Favoriscono riuso e manutenibilità

- Centralizzando la creazione, eventuali modifiche sono localizzate e controllate.

🔑 Idea chiave

Se nei pattern strutturali ci chiediamo:

“Come organizzo le classi tra loro?”

Nei pattern comportamentali:

“Come collaborano gli oggetti?”

Nei pattern creazionali ci chiediamo:

“Chi crea gli oggetti, e in che modo?”

Ed è proprio questa astrazione del processo di creazione che rende l'architettura più flessibile e meno accoppiata.

GoF: Pattern Structural

I pattern strutturali operano a livello di **composizione**:

- **cioè si occupano di come classi e oggetti vengono organizzati per costruire strutture più grandi, mantenendo il sistema flessibile.**

Il loro scopo generale è:

Comporre classi e oggetti in strutture più complesse senza aumentare inutilmente le dipendenze.

In pratica, **aiutano a collegare componenti già esistenti in modo elegante, evitando modifiche invasive.**

Quando si utilizzano?

I pattern strutturali risultano utili quando:

- **è necessario adattare classi esistenti a nuove interfacce (es. integrare codice legacy o librerie esterne)**
- **si vogliono comporre oggetti mantenendo basso il coupling**
- **si desidera fornire un'interfaccia semplificata verso un sottosistema complesso**
- **si devono aggiungere responsabilità senza modificare il codice esistente (principio Open/Closed)**

🔔 **Sono quindi fondamentali quando il sistema cresce e si rischia di creare dipendenze rigide tra moduli.**

✓ Vantaggi

- Favoriscono il riuso di componenti esistenti
- Riducono il coupling tra classi
- Permettono l'applicazione concreta del principio Open/Closed
- Rendono la struttura più modulare e facilmente estendibile

Se i pattern creazionali rispondono alla domanda “*Chi crea gli oggetti?*”, quelli strutturali rispondono a “*Come li organizzo tra loro?*”.

GoF: Pattern Behavioral

I pattern comportamentali **si concentrano sulle interazioni tra oggetti e sulla distribuzione delle responsabilità.**

Non si occupano della creazione né della struttura statica, **ma di come gli oggetti collaborano dinamicamente.**

Gestiscono principalmente:

- flussi di controllo
- algoritmi
- comunicazione tra oggetti
- distribuzione delle responsabilità

Quando si utilizzano?

Sono particolarmente utili quando:

- il comportamento varia in base al contesto
- più oggetti devono collaborare in modo coordinato
- si vogliono evitare strutture condizionali complesse (`if` , `switch`) distribuite nel codice
- è necessario separare chi richiede un'azione da chi la esegue

In altre parole, **aiutano a evitare logiche proceduralmente rigide e a favorire una progettazione più orientata agli oggetti.**

✓ Vantaggi

1. **Migliore separazione delle responsabilità all'interno di una collaborazione.**
2. Coupling contenuto, con conseguente:
 - **maggiore riutilizzo**
 - **maggiore manutenibilità**
 - **minore impatto delle modifiche**

Se i **pattern strutturali** lavorano sulla forma del sistema, i **pattern comportamentali** lavorano sul modo in cui il sistema si muove e reagisce.

E insieme ai pattern creazionali completano la visione dei 23 pattern del GoF, coprendo creazione, struttura e comportamento:

- **le tre dimensioni fondamentali della progettazione orientata agli oggetti.**

È una questione di principio

Nel mondo della progettazione software, oltre ai pattern, si sono affermati nel tempo alcuni **principi guida** che **aiutano a prendere decisioni più consapevoli.**

Non sono regole assolute né leggi matematiche.

La loro applicabilità dipende sempre dal contesto reale, dai vincoli del progetto e anche dalla maturità tecnica del team.

Come per i design pattern, vanno usati con criterio.

Di seguito alcuni tra i più noti.

YAGNI Principle

Acronimo di:

You Aren't Gonna Need It

“Non ne avrai davvero bisogno”

È un principio secondo cui:

- **lo sviluppatore non dovrebbe implementare funzionalità che non sono necessarie nel momento attuale.**

Spesso si tende ad anticipare esigenze future ipotetiche, aggiungendo codice “per sicurezza”.

✗ **Questo comporta però diversi rischi:**

- **maggiore tempo di progettazione**
- **maggiore tempo di sviluppo**
- **maggiore tempo di testing**
- **aumento della complessità**

⚡ **Oppure, scenario ancora peggiore:**

- **la funzionalità viene sviluppata superficialmente**
- **non viene documentata correttamente**
- **non viene testata adeguatamente**
- **peggiora la qualità complessiva del codice**

📄 **Il principio YAGNI invita quindi a concentrarsi sul problema reale, evitando sovra-progettazioni premature.**

KISS Principle

Acronimo di:

Keep It Simple and Short

oppure nella versione più nota e ironica

Keep It Simple, Stupid

Il principio afferma che:

- **la semplicità deve essere l'obiettivo chiave del design.**
Ogni complessità non strettamente necessaria dovrebbe essere eliminata.

✓ **Un sistema semplice:**

- **è più leggibile**

- **è più manutenibile**
- **è più facilmente estendibile**
- è meno soggetto a errori

L'idea non è banalizzare il problema, ma evitare soluzioni inutilmente sofisticate.

Il concetto è stato espresso in forme diverse anche da grandi pensatori come **Albert Einstein**, con la celebre frase:

“Rendi tutto il più semplice possibile, ma non più semplice.”

Quindi semplicità sì, ma senza perdere correttezza e completezza.

DRY Principle

Acronimo di:

Don't Repeat Yourself

noto anche come **Duplication Is Evil (DIE)**

È uno dei principi fondamentali dello sviluppo software.

Afferma che:

In un sistema, ogni singolo pezzo di informazione deve avere un'unica rappresentazione chiara e non ambigua .

La duplicazione può riguardare:

- **codice**
- **logica**
- **configurazioni**
- **regole di business**
- **strutture dati**

Se la stessa informazione è presente in più punti:

- **aumentano i rischi di incoerenza**
- **aumentano gli errori**
- **cresce il costo di manutenzione**

Il principio DRY è particolarmente rilevante nelle **architetture multi-tier**, dove i dati transitano tra livelli diversi (presentation, business, data layer).

☰ Riepilogo:

Possiamo sintetizzare questi principi in tre idee fondamentali:

1. **Fai solo ciò che serve** → YAGNI
2. **Rendilo semplice** → KISS
3. **Non ripetere ciò che hai già fatto** → DRY

Questi principi non sostituiscono i design pattern, ma li affiancano.

Se i pattern forniscono **strutture riusabili per risolvere problemi ricorrenti**, questi principi aiutano a mantenere il progetto:

- **essenziale**
- **chiaro**
- **coerente**
- **sostenibile nel tempo**

Ed è proprio dall'equilibrio tra principi e pattern che nasce un buon design.

Adapter (Pattern Strutturale)

Ora che abbiamo analizzato i diversi pattern di progettazione, analizziamo nel dettaglio uno dei pattern strutturali: il adapter.

Adapter è uno dei pattern strutturali del GoF e ha lo scopo di:

- **rendere compatibili classi con interfacce diverse, senza modificarne il codice originale.**

Problema:

Il problema tipico è questo:

- **abbiamo una classe già esistente**
- **la sua interfaccia non è compatibile con quella che il nuovo sistema si aspetta**
- **non possiamo (o non vogliamo) modificarla**

In altre parole:

dobbiamo convertire l'interfaccia di una classe in un'altra interfaccia nota al client.

⚠ **Attenzione:**

qui per *client* non si intende il client dell'architettura Client-Server, ma il codice che utilizza l'oggetto.

Esempio concettuale

Supponiamo di avere:

- un progetto PRJ1
- una classe `Impiegato` già funzionante

Ora nasce un nuovo progetto PRJ2 che deve riutilizzare la logica di `Impiegato`, ma:

- l'interfaccia richiesta da PRJ2 è diversa
- non vogliamo copiare il codice
- non vogliamo modificare la classe originale

Il problema è quindi di **incompatibilità di interfacce**, non di logica.

Soluzione

La soluzione consiste nel creare una **classe Adapter** che:

- **implementa l'interfaccia richiesta da PRJ2**
- **contiene (o estende) un oggetto `Impiegato`**
- **traduce le chiamate del nuovo sistema nei metodi della classe originale**

Concettualmente:



CSS

```
1 Client (PRJ2) → Adapter → Impiegato (PRJ1)
```

L'Adapter fa da **ponte** tra:

- **la nuova interfaccia attesa dal client**
- **la classe concreta da adattare**

Non modifica l'oggetto originale, ma lo incapsula e ne traduce l'uso.

Struttura logica semplificata

Abbiamo tre elementi:

1. **Target** → interfaccia attesa dal client
2. **Adaptee** → classe esistente (`Impiegato`)
3. **Adapter** → classe che collega Target e Adaptee

L'Adapter realizza una relazione:

- *is-a* rispetto al Target
- *has-a* rispetto all'Adaptee

Quindi non è corretto dire che l'Adapter ha una relazione *is-a* con `Impiegato`.
Piuttosto:

- implementa l'interfaccia richiesta
- contiene un riferimento all'oggetto da adattare

≡ Analogia dell'adattatore di corrente

Al fine di capire meglio questo pattern, possiamo usare l'analogia con l'adattatore delle prese di corrente.

Immaginiamo di avere un caricatore che ha la presa con spina americana ma stando in Italia non è possibile collegarla a nessuna presa di corrente.

La soluzione quindi è ovvia:

- usare un adattatore di presa di corrente.

L'adattatore:

- non modifica né la presa né la spina
- traduce il formato
- permette la collaborazione tra due standard incompatibili

Il pattern Adapter funziona esattamente allo stesso modo nel software.

🔍 Quando usare Adapter

- quando vogliamo riutilizzare una classe esistente
- quando l'interfaccia non è compatibile

- quando non possiamo modificare il codice originale
- quando vogliamo integrare librerie esterne

✓ Vantaggi

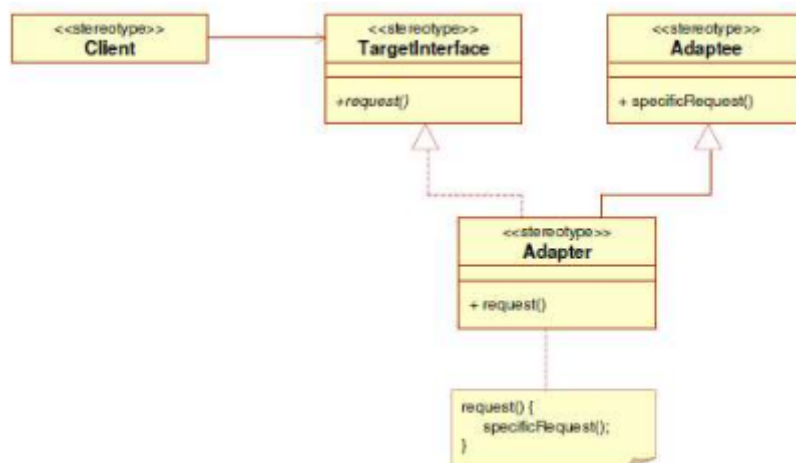
- favorisce il riuso
- evita duplicazione di codice
- riduce accoppiamento tra sistemi
- permette integrazione graduale tra componenti

Quindi:

- Adapter non cambia il comportamento della classe, cambia il modo in cui viene vista dall'esterno.

Diagramma Adapter versione Class

Ora analizziamo la versione con il diagramma delle classi:



L'idea è questa:

- esiste un **Client**
- il Client lavora con un'interfaccia chiamata **TargetInterface**
- esiste una classe già pronta (**Adaptee**) che però non estende, ne partecipa a nessun link di associazione con quell'interfaccia
- Per ovviare a questo problema si crea una classe **Adapter** che:
 - **estende** **Adaptee**
 - **implementa** **Target**

Struttura dei ruoli

Abbiamo quattro elementi principali:

1. Client

Il Client conosce solo l'interfaccia Target.

Non sa nulla della classe Adaptee.

Questo è fondamentale:

- **il client rimane disaccoppiato dalla classe concreta.**

2. Target (interfaccia)

Definisce il metodo atteso dal client, ad esempio:



Java

```
1 public interface Target {  
2     void request();  
3 }
```

È il contatto che il sistema si aspetta

3. Adaptee

È la classe già esistente, con una sua interfaccia diversa:



Java

```
1 public class Adaptee {  
2     public void specificRequest() {  
3         // logica esistente  
4     }  
5 }
```

Il problema è che il client non può chiamare specificRequest() direttamente.

4. Adapter

È la classe chiave.

Nella versione **class adapter**:

- ==estende Adaptee , invocando il metodo specificRequest() ==
- **implementa Target , overriding, e concretizzando, il metodo request e invocando la funzione specificRequest() nel corpo del metodo overrideato**

E concretizza il metodo richiesto dal client:



Java

```

1  public class Adapter extends Adaptee implements Target {
2
3      @Override
4      public void request() {
5          specificRequest(); // traduzione della chiamata
6      }
7  }
```

Quindi qui avviene la **mappatura**:



Sass (SCSS)

```

1  request()  →  specificRequest()
```

Quindi l'adattatore possiede metodi vecchi e nuovi, ovvero:

- **eredita i metodi di** Adaptee
- **implementa i metodi richiesti da** Target

Quindi l'Adapter espone:

- **la nuova interfaccia (verso il client)**
- **la vecchia funzionalità (tramite ereditarietà)**

Aspetto importante

Questa versione funziona bene quando:

- **possiamo usare l'ereditarietà**
- **non abbiamo vincoli di estensione (es. linguaggi con ereditarietà singola)**

Infatti:

La versione Class Adapter è meno flessibile rispetto alla versione Object Adapter, perché vincola l'adattatore a una specifica classe base.

☰ In sintesi

Nel Class Adapter:

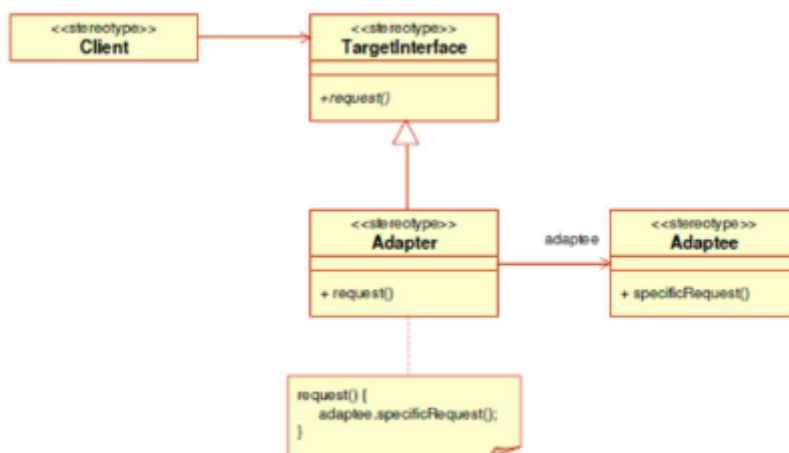
- il client parla con Target
- l'Adapter traduce la richiesta
- la traduzione avviene tramite ereditarietà
- la logica originale non viene modificata

In sostanza:

Adapter non cambia l'Adaptee, ma lo rende compatibile con il sistema che lo deve usare.

Diagramma Adapter versione Object

Nella versione Object Adapter, l'adattamento non avviene più tramite ereditarietà, ma tramite **composizione**.



In questo caso :

- si crea una nuova classe Adapter
- implementa l'interfaccia richiesta dal Client (Target)
- contiene al suo interno un'istanza di Adaptee
- delega a quell'oggetto le operazioni reali

Struttura dei ruoli

I ruoli in queste versione non cambiano rispetto alla versione Adapter Class, abbiamo sempre:

- Client
- TargetInterface (interfaccia attesa)
- Adaptee (classe esistente)
- Adapter

Ma cambia la relazione tra Adapter e Adaptee .

Differenza fondamentale rispetto alla versione Class

Nella versione class abbiamo la classe Adapter che estende la classe Adaptee , mentre nella versione Object abbiamo la classe Adapter che partecipa ad un link di associazione con responsabilità singola verso Adaptee .

Quindi non eredita ma incapsula.

Struttura tipica

Interfaccia Target



Java

```
1 public interface Target {  
2     void request();  
3 }
```

Classe Adaptee



Java

```
1 public class Adaptee {  
2     public void specificRequest() {  
3         // logica esistente  
4     }  
5 }
```

Object Adapter



Java

```
1 public class Adapter implements Target {  
2
```

```

3     private Adaptee adaptee;
4
5     public Adapter(Adaptee adaptee) {
6         this.adaptee = adaptee;
7     }
8
9     @Override
10    public void request() {
11        adaptee.specificRequest(); // delega
12    }
13 }

```

Qui il metodo `request()` **non eredita nulla**, ma:

- traduce la chiamata
- la inoltra all'oggetto interno

Quindi in questa versione l'adattatore possiede solo i metodi nuovi, ovvero:

- **I'Adapter non eredita i metodi di Adaptee**
- espone solo quelli definiti nell'interfaccia `TargetInterface`
- usa internamente l'oggetto `Adaptee` per svolgere il lavoro

Quindi:

- verso l'esterno → espone solo la nuova interfaccia
- verso l'interno → delega alla classe esistente

✓ Vantaggi della versione Object

È generalmente più flessibile perché:

- **non dipende dall'ereditarietà**
- **può adattare anche classi `final`**
- **può adattare più sottoclassi di `Adaptee`**
- **segue il principio “preferire composizione a ereditarietà”**

☰ Confronto tra queste due versioni

Class Adapter	Object Adapter
Usa ereditarietà	Usa composizione

Class Adapter	Object Adapter
Più rigido	Più flessibile
Eredita i metodi	Delega ai metodi
Vincolato alla classe base	Può lavorare con qualsiasi istanza

In sintesi, nella versione Object:

L'Adapter non è una sottoclasse dell'Adaptee,
ma un oggetto che lo incapsula e traduce le chiamate.

In termini architetturali, è la soluzione più pulita e più aderente ai principi di basso accoppiamento.

❓ **Class Adapter vs Object Adapter: quale scegliere**

la scelta tra **Class Adapter** e **Object Adapter** è principalmente **situazionale**, ma non arbitraria:

- **dipende dai vincoli del linguaggio, dalla struttura del sistema e dal livello di flessibilità richiesto.**

Quando preferire il *Class Adapter*

Abbiamo detto che l'Adapter in questa versione

- **estende l'** Adaptee
- **implementa il** Target

Si basa quindi sull'**ereditarietà multipla concettuale** (in Java: estensione di una classe + implementazione di un'interfaccia).

✓ **Convieni quando:**

- **Vuoi riutilizzare direttamente il comportamento dell'Adaptee**
- **Hai bisogno di ridefinire (override) alcuni metodi dell'Adaptee**
- **L'Adaptee è una classe concreta che puoi estendere**
- **Non hai bisogno di adattare più classi diverse**

⚠ **Limiti**

- Forte accoppiamento alla classe concreta
- In Java puoi estendere una sola classe
- Meno flessibile in caso di variazioni future

Quando preferire l'Object Adapter

Quindi abbiamo detto che questa versione non si basa sull'ereditarietà ma incapsula l'Adapter:

- implementa il `Target`
- contiene un'istanza di `Adaptee`

Qui si usa la composizione invece dell'ereditarietà.

✓ **Convienne quando:**

- Vuoi una soluzione più flessibile
- Devi adattare più classi diverse
- Non puoi o non vuoi estendere l'Adaptee
- Vuoi ridurre l'accoppiamento
- L'Adaptee è `final` o appartiene a librerie esterne

⚠ **Limiti**

- Non puoi fare override diretto del comportamento interno
- Leggera complessità in più nella delega delle chiamate

👁 **Nella pratica moderna Java, si tende quasi sempre a preferire Object Adapter, perché:**

- la composizione è più flessibile dell'ereditarietà
- rispetta meglio il principio *"favor composition over inheritance"*
- evita rigidità strutturali

Adapter

L'applicazione del **pattern Adapter** comporta una serie di effetti strutturali e progettuali che possono essere sia vantaggi sia aspetti da valutare con attenzione.

Vantaggi

- **Riutilizzo di classi esistenti**
Permette di integrare classi già sviluppate (legacy o di terze parti) senza modificarne il codice sorgente, rispettando il principio di *Open/Closed*.
- **Separazione tra client e implementazione concreta**
Il client dipende esclusivamente dall'interfaccia `Target` e non conosce la struttura interna della classe adattata (`Adaptee`).
Questo favorisce disaccoppiamento e maggiore manutenibilità.
- **Mascheramento dei dettagli implementativi**
Le differenze tra l'interfaccia attesa e quella reale vengono nascoste all'interno dell'adattatore.

Aspetti critici / Svantaggi

- **Maggiore complessità strutturale**
Introduce una classe aggiuntiva nel sistema, aumentando il numero di componenti da gestire.
- **Responsabilità concentrata sull'adattatore**
L'Adapter diventa responsabile della traduzione delle chiamate e, talvolta, anche della creazione e gestione dell'oggetto adattato (nella versione `Object`).
Se mal progettato, può trasformarsi in un punto di accoppiamento eccessivo.
- **Limitazioni nella versione `Class Adapter`**
Poiché utilizza l'ereditarietà, non può adattare contemporaneamente più classi (nei linguaggi a singola ereditarietà come Java).

Esempio concreto di caso d'uso del pattern Adapter

Supponiamo di avere un software gestionale per le assunzioni in un'azienda.

Nel sistema esistente (legacy) sono già presenti le classi:

- `Impiegato`
- `Manager` (che estende `Impiegato`)
- `Azienda`

Ora si introduce un nuovo requisito: il sistema deve lavorare con un nuovo tipo astratto chiamato `Dipendente`.

Nuovo requisito

Il nuovo modulo del sistema si aspetta oggetti che espongano i seguenti metodi:



Java

```
1 public String getNominativo();
2 public double getRetribuzioneAnnuua();
3 public int getAnniAnzianita();
```

Tuttavia, la classe `Impiegato` esistente:

- potrebbe avere nomi di metodi diversi
- potrebbe avere una struttura interna differente
- non può essere modificata (vincolo tipico nei sistemi legacy)

Problema

Il nuovo sistema lavora con il tipo `Dipendente`, ma i dati attuali sono rappresentati tramite oggetti `Impiegato`.

Non vogliamo:

- duplicare codice
- riscrivere `Impiegato`
- modificare classi già funzionanti

Soluzione: Adapter

Si introduce una classe `AdapterImpiegato` che:

- implementa l'interfaccia `Dipendente`
- contiene un riferimento a un oggetto `Impiegato`
- traduce le chiamate ai metodi richiesti in chiamate ai metodi reali dell'oggetto adattato

Classi Legacy

Classe `Impiegato`

Rappresenta un lavoratore generico dell'azienda.

È la **classe base concreta** su cui si fonda il modello attuale.

Implementa:



Java

```
1 public class Impiegato implements Comparable<Impiegato>
```

Quindi:

- è confrontabile (ordinabile)
- può essere inserita in strutture ordinate (TreeSet , Collections.sort , ecc.)



Java

```
1 package azienda;
2
3 import java.text.DateFormat;
4 import java.util.Date;
5
6 import java.util.Objects;
7
8 public class Impiegato implements Comparable<Impiegato> {
9     private final String nome;
10    private double salario;
11    private final Date dataAssunzione;
12
13    // La variabile statica non viene considerata nella wizard di
14    // Source → generate constructor using fields..
15
16    private static int contatore = 0;
17
18    public Impiegato(String nome, double salario, Date
19    dataAssunzione) {
20        this.nome = nome;
21        this.salario = salario;
22        this.dataAssunzione = dataAssunzione;
23        contatore++;
24    }
25
26    public String getNome() {
```



```

26         return nome;
27     }
28
29     public double getSalario() {
30         return salario;
31     }
32
33     public Date getDataAssunzione() {
34         return dataAssunzione;
35     }
36
37     public static int getContatore() {
38         return contatore;
39     }
40
41     public void setSalario(double salario) {
42         this.salario = salario;
43     }
44
45     @SuppressWarnings("deprecation")
46     public int getAnnoAssunzione() {
47         return this.dataAssunzione.getYear() + 1900;
48     }
49
50     public void incrSalario(double aumento) {
51         this.salario += aumento;
52     }
53
54
55     @Override
56     public String toString() {
57         DateFormat df =
58         DateFormat.getDateInstance(DateFormat.FULL);
59         String dFormattata = df.format(dataAssunzione);
60         return "Nome Dipendente = " + nome + ", salario = " +
61         salario + ", dataAssunzione = " + dFormattata;
62     }
63
64     // Ordina per nome per criterio lessografico
65     public int compareTo(Impiegato p) {
66         return this.getNome().compareTo(p.getNome());
67     }
68
69     @Override

```

```

68     public int hashCode() {
69         return Objects.hash(dataAssunzione, nome, salario);
70     }
71
72     @Override
73     public boolean equals(Object obj) {
74         if (this == obj) {
75             return true;
76         }
77         if (obj == null) {
78             return false;
79         }
80         if (getClass() != obj.getClass()) {
81             return false;
82         }
83         Impiegato other = (Impiegato) obj;
84         return Objects.equals(dataAssunzione,
other.dataAssunzione) && Objects.equals(nome, other.nome)
85             && Double.doubleToLongBits(salario) ==
Double.doubleToLongBits(other.salario);
86     }
87 }

```

La classe Manager

È una specializzazione di `Impiegato`.

Ed introduce il campo `private String segretaria;`.



Java

```

1  package azienda;
2  import java.util.Date;
3  public class Manager extends Impiegato {
4
5      private String segretaria;
6
7      public Manager(String nome, double salario, Date
dataAssunzione, String segretaria) {
8          super(nome, salario, dataAssunzione);
9          this.segretaria = segretaria;
10     }
11
12     public String getSegretaria() {
13         return segretaria;

```

```

14     }
15
16     public void setSegretaria(String segretaria) {
17         this.segretaria = segretaria;
18     }
19
20     // salario = 2_000
21
22     public Manager(String nome, Date dataAssunzione, String
segretaria) {
23         super(nome, 2_000 ,dataAssunzione);
24         this.segretaria = segretaria;
25     }
26
27     @Override
28
29     public void incrSalario(double aumento) {
30         Date oggi = new Date();
31         @SuppressWarnings("deprecation")
32         double bonus = 0.5 *(oggi.getYear()+1900 -
this.getAnnoAssunzione());
33         super.incrSalario(aumento + bonus);
34     };
35
36     @Override
37
38     public String toString() {
39         return super.toString() + ", segretaria = " + segretaria;
40     }
41 }

```

La classe Azienda

Rappresenta un aggregatore di `Impiegato` .



Java

```

1  package azienda;
2  import java.util.ArrayList;
3  public class azienda {
4      private String nome;
5      private ArrayList<Impiegato> dipendenti;
6      public azienda(String nome) {
7          super();

```

```

8         this.nome = nome;
9         this.dipendenti = new ArrayList<Impiegato>();
10    }
11
12    public String getNome() {
13        return nome;
14    }
15
16    public void setNome(String nome) {
17        this.nome = nome;
18    }
19
20    public ArrayList<Impiegato> getDipendenti() {
21        return dipendenti;
22    }
23
24    @Override
25    public String toString() {
26        String s = "Nome azienda: " + this.nome + "\n";
27        s+= "elenco dipendenti: \n";
28        for (Impiegato impiegato : dipendenti) {
29            s+= impiegato.toString() + "\n";
30        }
31        return s;
32    }
33
34    public void assume(Impiegato impiegato) {
35        this.dipendenti.add(impiegato);
36    }
37
38    public void incrSalarioPerTutti(double aumento) {
39
40        for (Impiegato impiegato : dipendenti) {
41            impiegato.incrSalario(aumento);
42        }
43    }
44 }

```

Osservazione Importante

Qui emerge un punto chiave:



Java

```
1 private ArrayList<Impiegato> dipendenti;
```

Azienda è accoppiata direttamente a:

- una classe concreta (Impiegato)
- una struttura concreta (ArrayList)

Questo riduce flessibilità.

Se domani volessimo:

- inserire un tipo diverso di lavoratore
- cambiare struttura dati

dovremmo modificare Azienda .

Implementazione dell'Adapter (versione Object)

Ora per fare in modo che Impiegato possa comunicare con Dipendente e viceversa scegliamo di implementare la versione Object, quindi:

Questo significa che:

- l'adattatore non estende la classe da adattare
- ma contiene un riferimento all'oggetto da adattare (composizione)

Quindi:

- Dipendente → rappresenta il Target (interfaccia attesa dal client)
- Impiegato → è l'Adaptee (classe legacy)
- AdapterImpiegato → è l'Adapter
- Il modulo che utilizza Dipendente → è il Client

1 Interfaccia Dipendente (Target)



Java

```
1 // NOTA: il package rispetto alle classi mostrate sopra è diverso
2 package adapter;
3 public interface Dipendente {
4     public String getNomativo();
5     public double getRetibuzioneAnnua();
6     public int getAnniAnzianita();
7 }
```

Definisce il **contratto** che il nuovo sistema si aspetta.

È importante osservare che:

- Dipendente **non conosce** Impiegato
- Impiegato **non conosce** Dipendente
- **Le due gerarchie sono indipendenti**

L'interfaccia stabilisce il **linguaggio comune**.

2 Classe AdapterImpiegato (Adapter)



Java

```
1  package adapter;
2  import java.time.LocalDate;
3  import azienda.Impiegato;
4
5  public class AdapterImpiegato implements Dipendente {
6      private Impiegato imp;
7      // 1 soluzione: inizializzo il campo impiegato nel
        costruttore
8      public AdapterImpiegato(Impiegato imp) {
9          super();
10         this.imp = imp;
11     }
12     // metodo getter
13     public Impiegato getImp() {
14         return imp;
15     }
16     // metodo setter
17     public void setImp(Impiegato imp) {
18         this.imp = imp;
19     }
20
21     // ----- Esercizio adattamento -----
22
23     @Override
24     public String getNomativo() {
25         // Implemento il metodo astratto dell'interfaccia Dipendente
26         // Restituisco il nominativo dell'oggetto impiegato
27         return this.imp.getNome();
28     }
29
```

```

30     @Override
31     public double getRetibuzioneAnnua() {
32         // TODO Auto-generated method stub
33         return this.imp.getSalario()*13;
34     }
35
36     @Override
37     // Implementazione metodo Astratto di Dipendente
38     public int getAnniAnzianita() {
39         // Da LocalDate prendo l'istante ed estraggo l'anno
40         // e lo salvo in una variabile
41         int annoAttuale = LocalDate.now().getYear();
42         // Restituisco il prodotto della sottrazione tra l'anno
attuale - l'anno di assunzione dell'oggetto imp:Impiegato
43         return annoAttuale - this.imp.getAnnoAssunzione();
44     }
45
46     @Override
47
48     public String toString() {
49
50         return "AdapterImpiegato :{ Impiegato : " + getImp() + ",
Nominativo : " + getNomativo() + ", Retibuzione annua : " +
getRetibuzioneAnnua() + ", Anni di anzianita : " +
getAnniAnzianita() + " anni." +" }";
51     }
52 }

```

Analisi

1. Implementazione del Target e composizione dell'Adaptee



Java

```

1  public class AdapterImpiegato implements Dipendente {
2      private Impiegato imp;

```

Qui avviene la parte fondamentale del pattern:

👉 **L'adapter implementa il Target**

👉 **L'adapter contiene l'Adaptee**

Questa è la struttura tipica dell'Object Adapter.

2. Costruttore:



Java

```
1 public AdapterImpiegato(Impiegato imp) {  
2     this.imp = imp;  
3 }
```

L'oggetto legacy viene iniettato nell'adapter.

Questo permette:

- riuso dell'oggetto esistente
- nessuna modifica al codice legacy
- nessuna duplicazione

Analisi dei metodi di adattamento

Ora analizziamo l'adattamento vero e proprio.

1. `getNominativo()`



Java

```
1 @Override  
2 public String getNominativo() {  
3     return this.imp.getNome();  
4 }
```

Mappatura diretta:

- **Target(Interfaccia Dipendente)**: `getNominativo()`
- **Adaptee(Classe Impiegato)**: `getNome()`

L'adapter traduce il nome del metodo.

2. `getRetribuzioneAnnuale()`



Java

```
1 @Override  
2 public double getRetribuzioneAnnuale() {  
3     return this.imp.getSalario()*13;  
4 }
```

Qui non c'è solo mappatura, ma anche **trasformazione logica**.

Il nuovo sistema richiede:

- **retribuzione annua**

Il legacy fornisce:

- **salario mensile**

L'adapter:

- **converte il dato**
- **aggiunge significato**
- **incapsula la logica di trasformazione**

Questo è un caso tipico di responsabilità dell'adattatore.

3. `getAnniAnzianita()`



Java

```
1 int annoAttuale = LocalDate.now().getYear();
2 return annoAttuale - this.imp.getAnnoAssunzione();
```

Anche qui:

- **Il legacy fornisce anno di assunzione**
- **Il Target richiede anni di anzianità**

L'adapter calcola la differenza.

Cosa accade architetturalmente

Abbiamo ottenuto:

- **Nessuna modifica alla classe** `Impiegato`
- **Nessuna modifica alla classe** `Azienda`
- **Compatibilità con il nuovo contratto** `Dipendente`

Il client potrà scrivere:



Java

```
1 Dipendente d = new AdapterImpiegato(new Impiegato(...));
```

E lavorare solo con `Dipendente`.

🔗 Punto concettuale importante

L'Adapter:

- isola il legacy
- centralizza la logica di trasformazione
- evita duplicazione
- evita copia-incolla
- evita modifiche invasive

Questo rispetta:

- Open/Closed Principle
- Single Responsibility Principle

Pattern Strategy

Il pattern **Strategy** nasce quando un sistema deve offrire più varianti di uno stesso algoritmo o comportamento, senza che tali varianti vengano inglobate rigidamente dentro la classe che le utilizza.

Immaginiamo un programma che debba:

- ordinare dati secondo criteri diversi,
- calcolare prezzi con politiche differenti,
- applicare logiche alternative in base al contesto.

Un approccio ingenuo consisterebbe nell'inserire all'interno della classe principale una serie di `if` o `switch` per distinguere i casi.

Tuttavia, questa soluzione presenta diversi problemi:

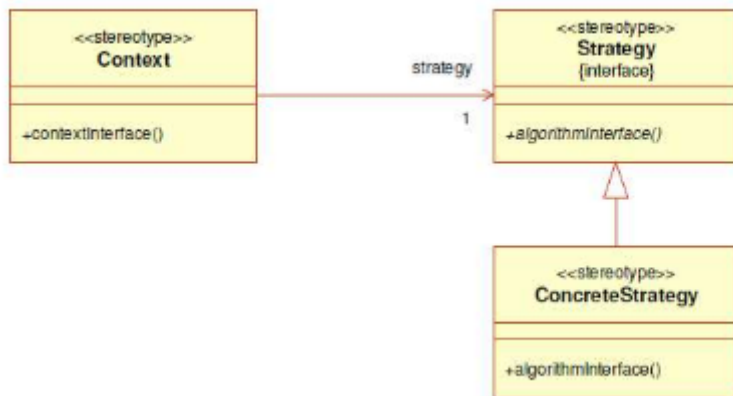
- il codice diventa meno leggibile,
- aumenta l'accoppiamento tra le componenti
- e ogni nuova variante richiede la modifica della classe esistente, violando il principio Open/Closed.

Il cuore del problema è questo:

- i comportamenti devono poter variare, ma senza modificare continuamente la classe che li utilizza.

L'idea alla base del pattern

La soluzione proposta dal pattern Strategy consiste nel **separare il comportamento dall'oggetto che lo utilizza**, incapsulandolo in classi autonome.



In pratica:

1. **Si definisce un'interfaccia comune (Strategy).**
2. **Si implementano diverse classi concrete che rappresentano le varie strategie.**
3. **La classe che utilizza l'algoritmo (detta Context) mantiene un riferimento all'interfaccia e delega ad essa l'esecuzione del comportamento.**

In questo modo il comportamento non è più "interno" alla classe, ma diventa intercambiabile.

Il Context non conosce i dettagli dell'algoritmo:

- **conosce solo l'interfaccia.**

Il ruolo del Client

È il **Client** a decidere quale strategia utilizzare, impostandola dinamicamente nel Context.

Questo significa che il comportamento dell'oggetto può cambiare a runtime, senza modificare il codice della classe principale.

Si passa quindi da:

- **comportamento statico e rigido**
a
- **comportamento dinamico e configurabile.**

Esempio concreto: TreeSet e Comparator

Un esempio molto chiaro è quello del `TreeSet`.

Supponiamo di creare un `TreeSet<Prodotto>`, è possibile passare nel costruttore un `Comparator`.



Java

```
1 TreeSet<Prodotto> prodotti = new TreeSet<>(new ProdComp());
```

Qui:

- `Comparator` è la Strategy
- `ProdComp` è la ConcreteStrategy
- `TreeSet` è il Context
- Il codice che istanzia il `TreeSet` è il Client

Il `TreeSet` non sa come confrontare i prodotti:

- sa solo che deve usare un oggetto che implementa `Comparator`.

Questo è Strategy puro.

✓ Vantaggi del Pattern

L'adozione di Strategy comporta diversi vantaggi:

- elimina strutture condizionali complesse,
- migliora la leggibilità del codice,
- rende il sistema estendibile,
- favorisce il riuso delle strategie.

Inoltre, si sfrutta il polimorfismo in modo non banale:

- non si cambia la gerarchia delle classi, ma si cambia l'algoritmo associato all'oggetto.

Un'osservazione sull'incapsulamento

È vero che il pattern comporta una leggera “apertura” dell'incapsulamento, perché parti di logica che prima erano interne alla classe vengono spostate

all'esterno.

Tuttavia questo non è uno svantaggio, bensì una scelta progettuale consapevole:

- **si sacrifica un po' di chiusura interna per ottenere maggiore flessibilità e manutenibilità.**

In sintesi, il pattern Strategy consente di trattare gli algoritmi come oggetti intercambiabili, rendendo il comportamento dinamico, configurabile e indipendente dalla classe che lo utilizza.